Project Report

on

“Cloud Fusion: Bridging KVM, VMware and AWS for the CivicSense Platform”

Submitted by

Mohammad Iqbal Patan

UID: 24MCC20081

MCA (CC & DevOps) · Semester 3 · Section 102/A

Under the guidance of

Shah Dhruv Samir

Subject: Cloud Virtualization | Subject Code: 25CAH-733

Class Group: 25CAH-733:P::GP-A

in partial fulfilment for the award of the degree of

MASTER OF COMPUTER APPLICATIONS
CLOUD COMPUTING & DEVOPS

University Institute of Computing (UIC)

Chandigarh University

October 2026

TABLE OF CONTENTS

Certificate **6**
Declaration **7**
Acknowledgement **8**
Abstract **9**
1. Introduction **10**
1.1 Background 10
1.2 Problem Statement 10
1.3 Proposed Solution 11
1.4 Scope of the Project 11
1.5 Organisation of the Report 12
2. Background and Project Objectives **13**
2.1 Need for Virtualisation 13
2.2 The Server Stack Before and After Virtualisation 13
2.3 Types of Virtualisation 13
2.4 Hypervisors and Ring Levels 14
2.5 Storage and Network Virtualisation 14
2.6 Cloud Migration 14
2.7 Project Objectives 14
3. System Requirements **16**
3.1 Hardware Requirements 16
3.2 Software Requirements 16
3.3 Functional Requirements 17
3.4 Non-Functional Requirements 17
4. Tools and Technologies Used **18**
4.1 KVM, QEMU and libvirt 18
4.2 cloud-init 18
5. System Design **19**
5.1 Architecture 19
5.2 Storage Design 19
5.3 Network Design 20
5.4 Migration Design 20
6. Implementation Steps **22**
6.1 Step 1: Fetch and Verify the Base Image 22
6.2 Step 2: Provision VMs as Linked Clones 22
6.3 Step 3: Deploy CivicSense into the VM 22
6.4 Step 4: Measure 22
6.5 Step 5: Virtual Networks 22
6.6 Step 6: Snapshots, Ballooning, TRIM and Managed Save 22
6.7 Step 7: Convert to VMware 23
6.8 Step 8: Plan the Move to AWS 23
7. Implementation and Code Explanation **24**

7.1 Project Structure....24
7.2 Provisioning Script....24
7.3 Isolated Network and NIC Hot-Plug....25
7.4 Conversion to VMware....25
7.5 AWS Planning....26
8. Testing and Results....27
8.1 Testing Strategy....27
8.2 Provisioning a VM....27
8.3 Hardware-Assisted and Emulated Virtualisation....28
8.4 CPU and Disk Performance....28
8.5 The Application in a VM and in a Container....29
8.6 Resource Use under Load....29
8.7 Storage Virtualisation....30
8.8 Network Virtualisation....30
8.9 Ballooning and Managed Save....31
8.10 From KVM to VMware....32
8.11 Planning the Move to AWS....33
8.12 Summary of Results....34
8.13 Defects Found and Fixed....35
9. Screenshots....36
10. Conclusion....37
11. Future Scope....38
References....39
Appendix A: Running and Demonstrating the Project....40

LIST OF FIGURES

S. No.	Figure	Page
1	Figure 5.1 – Cloud Fusion: KVM host, VMs, storage and networks, and the VMware and AWS targets	19
2	Figure 5.2 – Linked clones on one base image, a disk-only snapshot, and TRIM	20
3	Figure 5.3 – The NAT and isolated networks with the measured results	20
4	Figure 5.4 – From KVM to VMware and to EC2: two disk formats and the re-platform path	21
5	Figure 8.1 – Provisioning cf-app and what the guest sees	27
6	Figure 8.2 – The same VM without hardware assistance	28
7	Figure 8.3 – The same CPU and disk measurement in four environments	28
8	Figure 8.4 – Three rounds against the container and the KVM guest	29
9	Figure 8.5 – A disk-only snapshot before a bad upgrade, and the way back	30
10	Figure 8.6 – TRIM through the virtual disk returns 700 MiB to the host	30
11	Figure 8.7 – Throughput on both networks, NIC hot-plug and isolation	31
12	Figure 8.8 – Live memory change and suspend-to-disk	32
13	Figure 8.9 – Conversion to both VMDK formats, VMware's check and the boot test	32
14	Figure 8.10 – Instance types, Mumbai prices, the Debian AMI and the dry run	33
15	Figure 8.11 – Diagnosing the boot loop: what looped and what booted	35
16	Figure 9.1 – CivicSense served by the KVM virtual machine, with 120 demonstration complaints; the footer reads "Release c51ec9d9 (kvm)"	36

LIST OF TABLES

S. No.	Table	Page
1	Table 2.1 – Traditional and virtualised infrastructure	13
2	Table 2.2 – Types of virtualisation compared in this project	13
3	Table 3.1 – Hardware requirements	16
4	Table 3.2 – Software requirements	16
5	Table 3.3 – Functional requirements	17
6	Table 4.1 – Tools and technologies used	18
7	Table 5.1 – Virtual machines	19
8	Table 8.1 – Provisioning times	27
9	Table 8.2 – CPU throughput	28
10	Table 8.3 – CivicSense under load (8 clients, 800 requests per round)	29
11	Table 8.4 – VMware disk formats of cf-app	33
12	Table 8.5 – Candidate EC2 instances (Mumbai, on-demand Linux, read on 31 October 2026)	33
13	Table 8.6 – The seven-step migration model applied to CivicSense	34
14	Table 8.7 – Summary of measured results	34
15	Table 8.8 – Defects found and fixed	35

CERTIFICATE

This is to certify that the project report entitled **"Cloud Fusion: Bridging KVM, VMware and AWS for the CivicSense Platform"** submitted by **Mohammad Iqbal Patan** (UID: 24MCC20081), MCA (Cloud Computing & DevOps), Semester III, Section 102/A (class group 25CAH-733:P::GP-A), in partial fulfilment of the requirements for the award of the degree of Master of Computer Applications (Cloud Computing & DevOps) at Chandigarh University, is a record of bonafide work carried out by the student under my supervision and guidance as part of the course **Cloud Virtualization (Subject Code: 25CAH-733)** during the academic session 2026–2027.

The project involved building and measuring virtual machines on KVM for a real Django application: linked clones from a verified cloud image configured by cloud-init, a comparison of hardware-assisted virtualisation with emulation and containers, storage and network virtualisation with snapshots, isolated networks and TRIM, the conversion of a virtual disk to VMware formats, and the planning of its migration to Amazon EC2. It demonstrates the practical application of cloud virtualisation to a real-world problem.

To the best of my knowledge, the work embodied in this report has not been submitted, in part or in full, to any other university or institution for the award of any degree or diploma.

Shah Dhruv Samir
Project Guide
University Institute of Computing

Head of Department
University Institute of Computing
Chandigarh University

Date: _____

Place: Chandigarh University, Mohali, Punjab

DECLARATION

I, **Mohammad Iqbal Patan**, student of Master of Computer Applications (Cloud Computing & DevOps), Semester III, University Institute of Computing, Chandigarh University, hereby declare that the project report entitled **"Cloud Fusion: Bridging KVM, VMware and AWS for the CivicSense Platform"**, submitted for the course Cloud Virtualization (25CAH-733), is an authentic record of my own work carried out under the guidance of Shah Dhruv Samir.

All the code, configuration, measurements and screenshots presented in this report were produced while building, running and testing the project. Wherever the work of others has been used, it has been duly acknowledged in the text and listed in the references.

This report has not been submitted to any other university or institution for the award of any degree or diploma.

Date: _____
Place: Mohali, Punjab

Mohammad Iqbal Patan
UID: 24MCC20081

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all those who contributed to the successful completion of this project titled **“Cloud Fusion: Bridging KVM, VMware and AWS for the CivicSense Platform”**.

First and foremost, I would like to thank **Shah Dhruv Samir**, my project guide, for the valuable guidance, continuous support and encouragement throughout the course of this project. The suggestions and feedback I received played a key role in shaping both the project and this report.

I extend my heartfelt thanks to the faculty members of the University Institute of Computing, Chandigarh University, for providing the facilities, the curriculum in virtualisation and cloud computing, and a stimulating environment in which to carry out this work.

I am also grateful to the communities behind KVM, QEMU, libvirt, Debian and cloud-init. Their freely available software, images and documentation made it possible to build a complete virtualisation laboratory on a single laptop.

Lastly, I would like to thank my family, friends and classmates for their motivation, moral support and constructive feedback during the development of this project.

Mohammad Iqbal Patan
UID: 24MCC20081

ABSTRACT

Virtualisation lets one physical computer run many isolated virtual machines and is the foundation of every public cloud. This project, **Cloud Fusion**, moves the CivicSense web application onto virtual machines on **KVM** and connects that on-premises platform with **VMware** and **Amazon Web Services**, as the course project of Cloud Virtualization asks. Every step is a script, and every result was measured.

Virtual machines were built from Debian's official cloud image, checked by SHA-512, as **copy-on-write linked clones** configured on first boot by **cloud-init**: a new VM answered SSH after 15.3 s and was fully configured after 68.2 s, and three VMs fitted in about 1.1 GiB. The same measurements ran on the host, in a container, in a hardware-assisted KVM guest and in a guest emulated by QEMU's TCG. With hardware assistance the guest computed at host speed (405 against 408 MB/s of SHA-256) and served CivicSense as fast as a container; without it the same work ran 4.7 times slower and the kernel booted in 15.9 s instead of 3.2 s. VM-to-VM traffic reached 19.4 Gbit/s on the NAT network and 17.2 Gbit/s on an isolated network that cut a VM off from the internet. A disk-only snapshot taken in 0.1 s turned a broken upgrade into an 8.9 s rollback; TRIM through the virtual disk returned 700 MiB to the host; ballooning and suspend-to-disk worked on the running application.

The VM's disk was converted to VMware's streamOptimized (443 MB) and monolithicSparse (1.2 GB) formats, accepted by VMware's vmware-vdiskmanager and booted again. The migration to EC2 was planned with real data from the AWS APIs: Mumbai prices, the official Debian 13 AMI of the same build, and a successful launch dry run, leading to a t3.small at about 17 USD a month for the measured CPU-bound workload. The defects met on the way – a boot loop caused by a VM without a video device, a snapshot that referred to a deleted file, and a merge into the wrong disk image – were diagnosed from evidence and fixed in the scripts.

The scripts and this report are available at https://github.com/iPatan7/cloud-fusion.

Keywords: Virtualisation, KVM, QEMU, libvirt, Hypervisor, Hardware-Assisted Virtualisation, Emulation, cloud-init, Linked Clones, Snapshots, Thin Provisioning, Network Virtualisation, VMware, VMDK, AWS EC2, Cloud Migration.

CHAPTER 1: INTRODUCTION

1.1 Background

For decades every server application ran on its own physical machine. A typical data centre filled racks with servers that were busy for a few minutes a day and idle the rest of the time, each with its own power supply, its own operating system to patch and its own spare parts. Adding a new application meant ordering hardware and waiting weeks.

Virtualisation breaks the link between software and hardware. A thin layer, the **hypervisor**, divides one physical computer into several **virtual machines** (VMs), each with its own virtual processors, memory, disks and network cards, and each running its own operating system as if it owned the hardware. Since processors gained hardware support for virtualisation (Intel VT-x, AMD-V), a VM runs ordinary instructions directly on the processor at nearly full speed; only privileged operations pass through the hypervisor. Storage and networks are virtualised in the same way: virtual disks are files that can be copied, cloned and snapshotted, and virtual switches connect VMs that never touch a cable.

Cloud computing is built on this foundation: an EC2 instance on Amazon Web Services is a virtual machine on a hypervisor in Amazon's data centre. Moving workloads between a company's own virtualised servers – often VMware – and a public cloud is therefore one of the central tasks of a cloud engineer, and the course project of Cloud Virtualization is named after it: *Cloud Fusion: Bridging VMware, KVM and AWS*.

1.2 Problem Statement

The application used in this project is **CivicSense**, a civic complaint platform written in Python with Django, developed in Technical Training-II and already containerised and delivered through a CI/CD pipeline in other courses. To run it on virtual machines – on an organisation's own servers or in the cloud – several questions have to be answered with measurements rather than assumptions:

- **How are VMs built reproducibly?** A VM installed by hand from an ISO image cannot be recreated identically; a cloud needs VMs that configure themselves.
- **What does virtualisation cost?** How much slower are the processor, the disk, the network and the application inside a VM than on the host or in a container – with and without hardware assistance?
- **How are storage and networks virtualised in practice?** Can one disk image serve many VMs, can a VM be snapshotted and rolled back, can a network be isolated without new hardware?

- **Can a VM move between hypervisors?** Does a disk built on KVM work for VMware, and what format does a cloud import need?
- **What would the cloud cost?** Which EC2 instance fits the measured workload, at what price, and which migration path is sensible?

The problem addressed is therefore: **how can CivicSense be moved onto virtual machines in a reproducible way, with measured performance and resource costs, and prepared for both a VMware environment and Amazon EC2?**

1.3 Proposed Solution

The project, *Cloud Fusion*, uses one machine with KVM and libvirt as the on-premises virtualisation platform and connects it to the two other worlds:

- **KVM**: VMs built from Debian's official cloud image as **copy-on-write linked clones**, configured on first boot by **cloud-init**, on a NAT network and an **isolated network**, with snapshots, live memory ballooning, TRIM and suspend-to-disk; CivicSense deployed inside a VM;
- **comparison**: the same CPU, disk and application measurements on the host, in a container, in a KVM guest and in a guest **without hardware assistance** (QEMU's TCG emulator);
- **VMware**: the VM's disk converted to the two main VMDK formats, checked with VMware's own `vmware-vdiskmanager`, described in a `.vmx` file and booted again;
- **AWS**: instance types, prices and the matching official Debian image read from the AWS APIs, a launch checked with a dry run, and both migration paths – re-platform and VM Import – prepared without creating anything.

1.4 Scope of the Project

All results were measured on 31 October 2026 on the author's laptop (Ubuntu 26.04.1, Intel Core i7-10510U, QEMU 10.2.1, libvirt 12.0.0, Docker 29.9.0). The AWS part uses only read-only calls and dry runs, so it costs nothing; actually launching instances or importing the image is left as a documented next step. VMware Workstation 26.0.0 is installed on the machine, but its kernel modules are not built for the running kernel and building them requires administrator rights, so VMware is covered through its disk formats and tools, not by running VMs in it.

The scripts, configuration and this report are published at **https://github.com/iPatan7/cloud-fusion**.

1.5 Organisation of the Report

Chapter 2 explains the virtualisation concepts and lists the objectives. Chapter 3 and Chapter 4 describe the requirements and the tools. Chapter 5 presents the architecture, storage, network and migration design. Chapter 6 lists the implementation steps and Chapter 7 explains the main scripts. Chapter 8 reports every measurement and the defects found. Chapter 9 shows the application running in the VM, followed by the conclusion, future scope, references and a step-by-step guide in Appendix A.

CHAPTER 2: BACKGROUND AND PROJECT OBJECTIVES

2.1 Need for Virtualisation

A physical server is sized for its peak load and spends most of its time far below it. Virtualisation consolidates many such workloads on fewer machines and adds abilities that physical servers do not have: a VM can be created from a template in a minute, snapshotted before a risky change, moved to another host, and given more memory without opening a case. Table 2.1 compares the two.

Table 2.1 – Traditional and virtualised infrastructure

Aspect	Traditional (physical)	Virtualised
Provisioning	Order, install and cable hardware: weeks	Clone a template: about a minute (measured 15 s to SSH)
Utilisation	One application per server, mostly idle	Many VMs share one host
Isolation	Physical separation	Hypervisor separates VMs; virtual networks separate traffic
Recovery	Reinstall or restore from backup	Snapshot revert, suspend and resume
Portability	Tied to the hardware model	Disk image moves between hosts, hypervisors, clouds
Cost	Capital cost per server, power, space	Fewer hosts; in the cloud, pay per hour

2.2 The Server Stack Before and After Virtualisation

Without virtualisation the stack is hardware, one operating system and its applications. With a hypervisor a new layer appears between hardware and operating systems: each VM sees virtual hardware – a virtual CPU, a virtual disk controller, a virtual network card – and runs its own kernel on it. KVM makes the Linux kernel itself the hypervisor: the `kvm` module uses the processor's VT-x extensions to run guest code, QEMU emulates the virtual devices, and libvirt manages VMs, storage pools and networks through one interface (`virsh`, `virt-install`).

2.3 Types of Virtualisation

Table 2.2 – Types of virtualisation compared in this project

Type	How it works	Here
Full virtualisation, hardware assisted	Guest runs unmodified; the CPU's VT-x executes guest code directly, the hypervisor handles privileged operations	KVM guests cf-app and cf-peer
Para-virtualisation (devices)	The guest uses drivers that know they are virtual and talk to the host efficiently	virtio disk and network in all KVM guests

Type	How it works	Here
Emulation (simulation)	Every guest instruction is translated in software; no hardware help needed	cf-tcg: QEMU's TCG
OS-level virtualisation	Processes isolated by the host kernel (namespaces, cgroups); no guest kernel	Docker container baseline

2.4 Hypervisors and Ring Levels

On x86 processors privilege is organised in rings: the kernel runs in ring 0, applications in ring 3. A guest kernel also expects ring 0, which classic x86 could not give it safely. Early hypervisors used binary translation or para-virtualised kernels; today VT-x adds a *root mode* for the hypervisor (often called ring -1) and a *non-root mode* in which the guest kernel runs in its own ring 0. A **type 1** hypervisor runs directly on the hardware (VMware ESXi, Xen, and KVM, which is part of the Linux kernel); a **type 2** hypervisor runs as an application on a host operating system (VMware Workstation, VirtualBox).

2.5 Storage and Network Virtualisation

Storage virtualisation separates the disk a VM sees from where the data lives. A virtual disk is a file such as **qcow2** (QEMU) or **VMDK** (VMware) that is *thin-provisioned* (it grows as data is written) and can have a **backing file**: a VM's own disk records only its differences from a shared read-only base image, which makes **linked clones** and **snapshots** cheap. **Network virtualisation** connects VMs through software switches (Linux bridges) with NAT, DHCP and DNS provided by the host, and separates traffic with isolated networks, the software counterpart of a VLAN.

2.6 Cloud Migration

Moving a workload into a cloud follows one of a few broad approaches: **re-host** ("lift and shift": move the VM as it is), **re-platform** (rebuild it on the cloud's own images with small changes) or **re-architect**. The seven-step model of migration into a cloud – assessment, isolation, mapping, re-architecting, augmentation, testing and optimisation – gives the order of work; Section 8.11 applies it to CivicSense. The suitability of a workload depends on its CPU, memory, storage and network profile, which is why the workload was measured before choosing an instance type.

2.7 Project Objectives

The objectives of the project are:

- To build VMs on KVM **reproducibly** from an official cloud image, as linked clones configured by cloud-init, and to measure how long provisioning takes.
- To **measure the cost of virtualisation** – CPU, disk, network and a real web application – on the host, in a container, in a hardware-assisted VM and in an emulated VM.
- To demonstrate **storage virtualisation**: thin provisioning, linked clones, snapshots and TRIM, with measured sizes.
- To demonstrate **network virtualisation**: NAT and isolated networks and NIC hot-plug, with measured throughput.
- To move a VM's disk to **VMware** formats and prove that the converted disk works.
- To plan the migration to **AWS** with real instance data, prices and a permission check, and to choose an instance from the measured workload.
- To record the **defects** met on the way and how they were diagnosed.

CHAPTER 3: SYSTEM REQUIREMENTS

The "used in this project" columns record the actual machine and versions on which every result in Chapter 8 was measured.

3.1 Hardware Requirements

Table 3.1 – Hardware requirements

Component	Minimum	Used in this project
Processor	64-bit with VT-x or AMD-V	Intel Core i7-10510U, 4 cores / 8 threads, VT-x
Memory	8 GB	about 15 GB (14 GiB usable); VMs with 0.75–1.5 GiB each
Storage	20 GB free	NVMe SSD; only 1–2.8 GB free on the system partition, which shaped several decisions
Network	Internet for images and packages	Wi-Fi; VMs reach it through NAT

3.2 Software Requirements

Table 3.2 – Software requirements

Software	Version	Purpose
Host operating system	Ubuntu 26.04.1 LTS, kernel 7.0.0-38	Virtualisation host
QEMU / KVM	10.2.1	Device emulation, TCG, hardware-assisted VMs
libvirt / virt-install	12.0.0 / 5.1.0	VM, storage pool and network management
Guest image	Debian 13 generic cloud image (26 Oct 2026), kernel 6.12.113	Base image of every VM
cloud-init	25.1.4 (in the guest)	First-boot configuration
Docker	29.9.0	Container baseline
VMware Workstation	26.0.0 build 25388281	vmware-vdiskmanager; target format
AWS CLI	2.36.13	EC2 and Price List APIs (read-only)
iperf3	3.18	Network throughput
Python / Django (in the VM)	3.13.5 / 6.1.2	CivicSense with gunicorn 26.2.0 and SQLite

3.3 Functional Requirements

Table 3.3 – Functional requirements

ID	Requirement
FR1	A VM must be created from the base image with one command, without manual installation, and be reachable over SSH with a key.
FR2	Several VMs must share one base image; each stores only its own changes.
FR3	The same measurement must run unchanged on the host, in a container and in each VM.
FR4	A VM must be able to join an isolated network at run time and lose its internet access when its NAT interface is removed.
FR5	A VM must be snapshotted before a risky change and returned to the snapshot.
FR6	The VM's disk must be converted to VMware formats and checked by VMware's tool.
FR7	The AWS plan must be produced without creating resources or costs.

3.4 Non-Functional Requirements

- **Reproducibility** – every step is a script in the repository; the base image is verified by checksum.
- **Honest measurement** – repeated runs, the same software in every environment where possible, and differences explained rather than hidden.
- **Safety** – no resources created in the AWS account; VMs reachable only from the host; SSH keys only.
- **Economy** – thin provisioning and clean-up, because the system disk had little free space.

CHAPTER 4: TOOLS AND TECHNOLOGIES USED

Table 4.1 lists the tools and the role each played.

Table 4.1 – Tools and technologies used

Tool	Role	How it is used
KVM (kernel module)	Hypervisor	Runs guest code with VT-x
QEMU	Machine emulator	Virtual devices for KVM guests; complete CPU emulation (TCG) for cf-tcg
libvirt, virsh	Management	VMs, the storage pool, networks, snapshots, ballooning, managed save
virt-install	Provisioning	Creates VMs from a disk image with a cloud-init seed
qemu-img	Disk images	Information, conversion to VMDK, size measurement
cloud-init	First-boot configuration	User, SSH key, packages; also works on VMware and EC2
Docker	Container baseline	The same measurements without a guest kernel
vmware-vdiskmanager	VMware disk tool	Checks the converted VMDK
AWS CLI	Cloud planning	Instance types, prices, AMIs, RunInstances dry run
iperf3, openssl, dd	Measurement	Network throughput, CPU throughput, disk throughput
xorriso	Seed images	Builds the cloud-init ISO for virt-install
curl, awk, Python	Glue	HTTP load generator, timing and parsing in the scripts

4.1 KVM, QEMU and libvirt

The three work as layers. The `kvm` kernel module turns Linux into a type 1 hypervisor; QEMU is the user-space process that represents one VM, emulates its devices and uses `/dev/kvm` to run its CPUs; libvirt runs QEMU with a consistent configuration, manages storage pools and virtual networks and offers one API for everything. Without `/dev/kvm`, QEMU falls back to its Tiny Code Generator (TCG), which translates guest instructions in software – the "simulation" form of virtualisation.

4.2 cloud-init

cloud-init is the standard first-boot agent of cloud images. On its first start it looks for configuration data – here an ISO image labelled `cidata` attached by virt-install, on EC2 the instance metadata service – and creates users, installs SSH keys and packages. Because the same file works on KVM, VMware and EC2, it is the key to rebuilding the same server on any of the three.

CHAPTER 5: SYSTEM DESIGN

5.1 Architecture

Figure 5.1 shows the whole setup. On the host, libvirt manages one storage pool with the shared base image and the VMs' overlays, two virtual networks, and three VMs: `cf-app` runs CivicSense, `cf-peer` is the partner for network tests, and `cf-tcg` is the same VM without hardware assistance. A Docker container serves as the baseline without a guest kernel. On the right are the two targets: VMware, reached through disk conversion, and AWS, reached through its APIs.

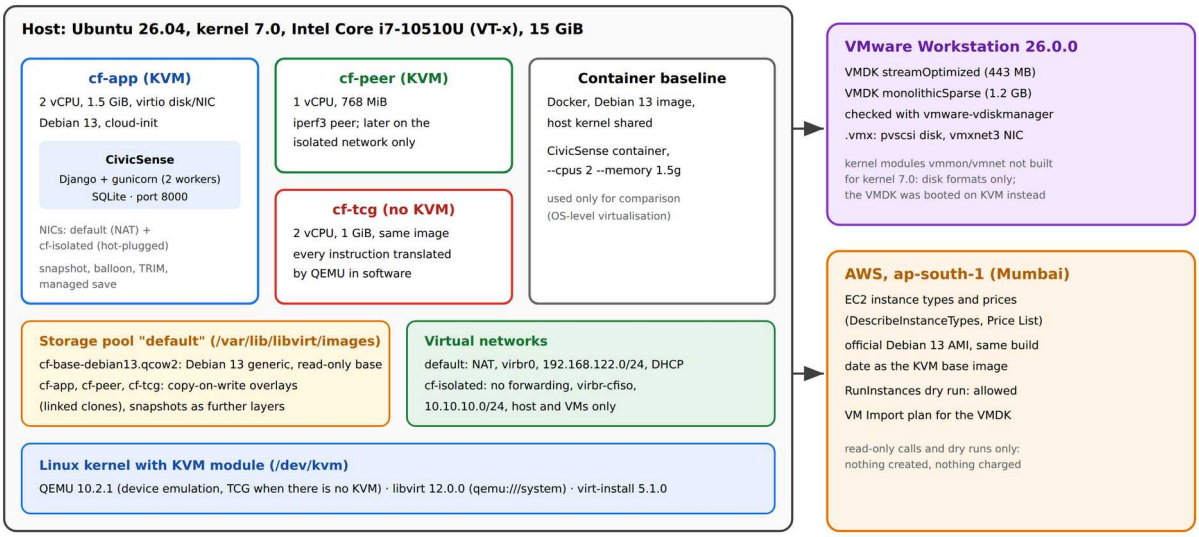


Figure 5.1 – Cloud Fusion: KVM host, VMs, storage and networks, and the VMware and AWS targets

Table 5.1 – Virtual machines

VM	vCPU	Memory	Acceleration	Role
cf-app	2	1.5 GiB	KVM	CivicSense; benchmarks; snapshots, balloon, TRIM, conversion
cf-peer	1	768 MiB	KVM	Network partner; isolated-network test; full snapshot
cf-tcg	2	1 GiB	none (TCG)	Emulation comparison

5.2 Storage Design

All VMs share one read-only base image, the Debian 13 generic cloud image, checked against Debian's SHA-512 list. Each VM gets a qcow2 overlay with the base as backing file (Figure 5.2): reads of unchanged blocks fall through to the base, writes go to the overlay. The "generic" image rather than the smaller "genericcloud" one was chosen because it contains the drivers for all three targets: virtio for KVM, vmxnet3 and pvscsi for VMware, ENA and NVMe for EC2.

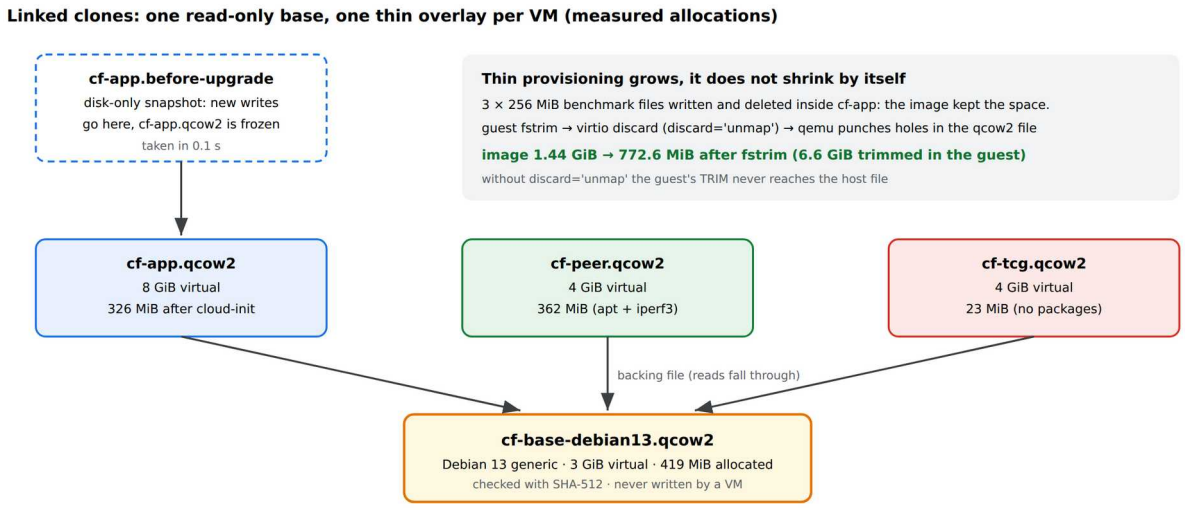


Figure 5.2 – Linked clones on one base image, a disk-only snapshot, and TRIM

5.3 Network Design

The default libvirt network is a NAT network on the bridge `virbr0` with DHCP and DNS from the host. A second network, `cf-isolated`, has no forwarding at all: VMs on it reach each other and the host, nothing else (Figure 5.3). It stands for an internal network such as a database VLAN. Interfaces are added to and removed from running VMs (hot-plug), so the topology can change without restarting anything.

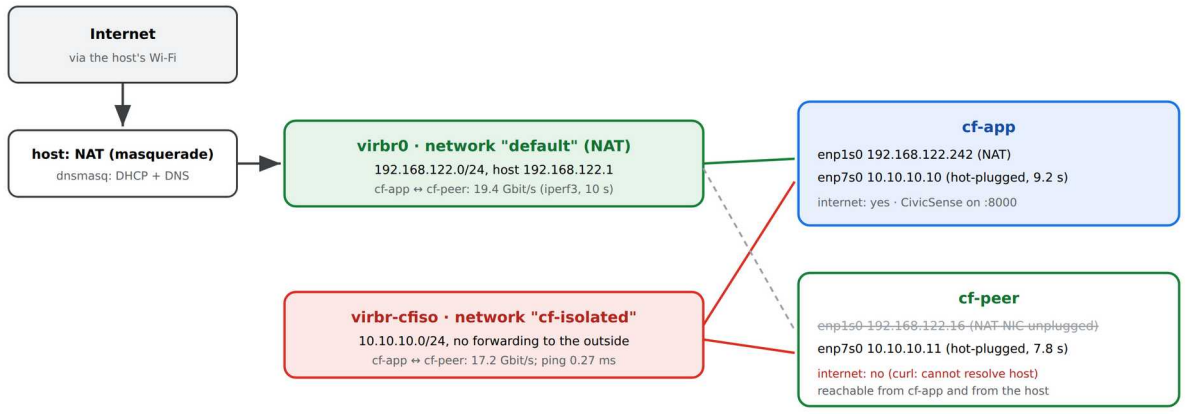


Figure 5.3 – The NAT and isolated networks with the measured results

5.4 Migration Design

Figure 5.4 shows the three ways out of KVM. The disk itself can be converted to VMware's Workstation format or to the compressed streamOptimized format that OVA files and AWS VM Import use. Alternatively the VM is not moved at all but rebuilt: the same cloud-init file and the same deployment script applied to the official Debian image on the target platform.

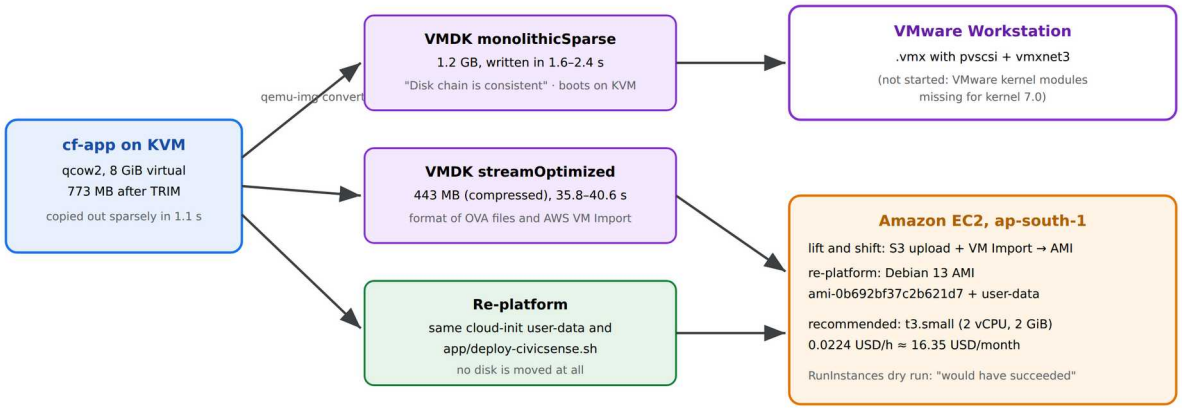


Figure 5.4 – From KVM to VMware and to EC2: two disk formats and the re-platform path

CHAPTER 6: IMPLEMENTATION STEPS

Every step is a script in the repository; the order below is the order in which they were run.

6.1 Step 1: Fetch and Verify the Base Image

`kvm/fetch-base.sh` downloads the Debian 13 generic image (419 MB in 61 s), checks it against Debian's SHA-512 list and uploads it into the libvirt pool `default`.

6.2 Step 2: Provision VMs as Linked Clones

`kvm/provision.sh` creates an overlay volume backed by the base image, fills in the cloud-init template with the host name, the lab SSH key and the packages, runs `virt-install`, and waits for an address, for SSH and for the end of cloud-init, printing each time.

```
kvm/provision.sh cf-app --memory 1536 --vcpus 2 --disk 8 --packages "python3-venv
iperf3"
kvm/provision.sh cf-peer --memory 768 --vcpus 1 --disk 4 --packages iperf3
kvm/provision.sh cf-tcg --tcg --memory 1024 --vcpus 2 --disk 4
```

6.3 Step 3: Deploy CivicSense into the VM

`app/deploy-civicsense.sh` streams the application code from the civicsense-devops repository into the VM, builds a Python virtual environment, creates the SQLite database with the demonstration data and installs gunicorn as a systemd service.

6.4 Step 4: Measure

`bench/cpu-disk.sh` runs the same OpenSSL and `dd` measurements in each environment; `bench/Dockerfile` builds the Debian 13 container used as the baseline; `bench/http.py` loads the application with eight parallel clients.

6.5 Step 5: Virtual Networks

`kvm/cf-isolated.xml` defines the isolated network; `kvm/attach-nic.sh` hot-plugs a NIC into a running VM and configures it in the guest with netplan.

6.6 Step 6: Snapshots, Ballooning, TRIM and Managed Save

`kvm/snapshot-test.sh` and `kvm/snapshot-app.sh` take a full and a disk-only snapshot around a deliberate bad change; ballooning, TRIM and managed save are single `virsh` commands, recorded with their timings.

6.7 Step 7: Convert to VMware

`convert/to-vmware.sh` copies the VM's disk out of the pool, converts it to both VMDK formats, checks the result with `vmware-vdiskmanager`, boots it, and writes a `.vmx` file.

6.8 Step 8: Plan the Move to AWS

`aws/plan.sh` reads instance types, prices and the newest official Debian 13 AMI for Mumbai and dry-runs `RunInstances`; `aws/VM-IMPORT.md` describes both migration paths.

CHAPTER 7: IMPLEMENTATION AND CODE EXPLANATION

7.1 Project Structure

Repository layout (github.com/iPatan7/cloud-fusion)

```
cloud-fusion/
├── kvm/
│   ├── fetch-base.sh         Debian image, SHA-512 check, pool upload
│   ├── provision.sh          linked clone + cloud-init + virt-install
│   ├── user-data.tmpl        cloud-init configuration
│   ├── ssh.sh                SSH to a VM with the lab key
│   ├── cf-isolated.xml       isolated libvirt network
│   ├── attach-nic.sh         NIC hot-plug + netplan in the guest
│   └── snapshot-*.sh         full and disk-only snapshot tests
├── app/deploy-civicsense.sh  CivicSense into a VM
├── bench/                    cpu-disk.sh, Dockerfile, http.py
├── convert/to-vmware.sh      VMDK formats, checks, .vmx
├── aws/                      plan.sh, VM-IMPORT.md
└── README.md
```

7.2 Provisioning Script

kvm/provision.sh (the clone and the VM)

```
# linked clone: an overlay whose backing file is the shared base image
$V vol-create-as default "$NAME.qcow2" "${DISK}G" --format qcow2 \
   --backing-vol "$BASE" --backing-vol-format qcow2 >/dev/null


T0=$(now)
virt-install --connect qemu:///system --name "$NAME" --memory "$MEM" --vcpus "$CPUS" \
  --virt-type "$VIRT" --osinfo debian13 --import \
  --disk "vol=default/$NAME.qcow2,bus=virtio" --network "network=$NET,model=virtio" \
  --cloud-init "user-data=$CF/seed/$NAME/user-data,meta-data=$CF/seed/$NAME/meta-data" \
  --graphics none --video vga --noautoconsole --quiet
# --video vga: without any video device ("--graphics none" alone) GRUB in the Debian
# 13 cloud image resets the VM while loading the kernel, about once a second.
```

`vol-create-as … --backing-vol` creates the linked clone. `--import` tells virt-install that the disk already contains an installed system, and `--cloud-init` builds the seed ISO. `--video vga` looks unnecessary for a VM without a screen, but without it the VM never boots (Section 8.13).

kvm/user-data.tmpl

```
#cloud-config
# First-boot configuration of every Cloud Fusion VM. kvm/provision.sh fills in
# @HOSTNAME@, @SSH_KEY@ and @PACKAGES@. The same file works unchanged on VMware and
# on AWS EC2, which also run cloud-init on first boot.
hostname: @HOSTNAME@
users:
  - name: civic
    groups: [sudo]
    shell: /bin/bash
    sudo: "ALL=(ALL) NOPASSWD:ALL"
    lock_passwd: true
    ssh_authorized_keys:
      - @SSH_KEY@
ssh_pwauth: false
package_update: @PACKAGE_UPDATE@
packages: [@PACKAGES@]
```

7.3 Isolated Network and NIC Hot-Plug

kvm/cf-isolated.xml

```
<!-- A libvirt network with no <forward> element: the VMs on it can reach each
     other and the host (10.10.10.1), but nothing outside the machine. It plays
     the role of a separate VLAN for traffic that must stay internal, such as the
     database network of an application.
       virsh net-define kvm/cf-isolated.xml && virsh net-start cf-isolated -->
<network>
  <name>cf-isolated</name>
  <bridge name='virbr-cfiso' stp='on' delay='0'/>
  <ip address='10.10.10.1' netmask='255.255.255.0'>
    <dhcp>
      <range start='10.10.10.10' end='10.10.10.50'/>
    </dhcp>
  </ip>
</network>
```

kvm/attach-nic.sh (host and guest side)

```
virsh -q -c qemu:///system attach-interface "$VM" network "$NET" --model virtio --mac "$MAC" --
live --config
"$HERE/ssh.sh" "$VM" "set -e
until ip -br link | grep -q '$MAC'; do sleep 0.5; done
printf 'network:\n  version: 2\n  ethernets:\n    $NET:\n      match: {macaddress: \"$MAC\"}\n
dhcp4: true\n      dhcp4-overrides: {use-routes: false}\n' | sudo tee /etc/netplan/60-$NET.yaml
>/dev/null
sudo chmod 600 /etc/netplan/60-$NET.yaml
sudo netplan apply
until ip -4 -br addr | grep -q '10\.10\.10\.'; do sleep 0.5; done
ip -4 -br addr | grep '10\.10\.10\.'"
```

7.4 Conversion to VMware

convert/to-vmware.sh (the conversion loop)

```
# A flat VMDK can need as much space as the data in the disk: refuse to start a
# conversion unless that much (plus 300 MiB of margin) is free.
# (qemu-img measure gives the size of the data once uncompressed)
need=$(( $(qemu-img measure --output=json -O qcow2 "$OUT/source.qcow2" | python3 -c 'import
json,sys; print(json.load(sys.stdin)["required"])') / 1048576 + 300 ))
echo "space needed per conversion: $need MiB"
for fmt in ${FORMATS:-streamOptimized monolithicSparse}; do
  f="$OUT/$VM-$fmt.vmdk"
  free=$(df -m --output=avail "$OUT" | tail -1)
  if [ "$free" -lt "$need" ]; then echo "--- $fmt: skipped, $free MiB free, $need MiB needed";
continue; fi
  t=$(now)
  qemu-img convert -O vmdk -o subformat=$fmt,adapter_type=lsilogic "$OUT/source.qcow2" "$f"
  echo "--- $fmt: $(du -h "$f" | cut -f1) on disk, written in $(since "$t") s"
  qemu-img info "$f" | grep -E 'file format|virtual size'
  if [ $fmt = monolithicSparse ]; then
    echo "vmware-vdiskmanager -e: $(vmware-vdiskmanager -e "$f" 2>&1 | grep -v '^\s*$' | tail
-1)"
    vmware-vdiskmanager -R "$f" >/dev/null 2>&1; echo "vmware-vdiskmanager -R (check and repair):
exit $?, nothing to repair when 0"
    log="$OUT/boot-$fmt.log"
    t=$(now)
    timeout -s KILL 60 qemu-system-x86_64 -enable-kvm -cpu host -m 1024 -display none -vga std \
      -serial file:"$log" -drive file="$f",format=vmdk,if=virtio,snapshot=on >/dev/null 2>&1 ||
true
    if grep -q 'login:' "$log"; then
      echo "booted from the VMDK on KVM: login prompt reached ($(grep -m1 -o 'Debian GNU/Linux
[0-9]*' "$log"))"
    else
      echo "boot from the VMDK did not reach a login prompt within 60 s"
    fi
  fi
  [ "${KEEP:-0}" = 1 ] || rm -f "$f"
done
```

7.5 AWS Planning

aws/plan.sh (price and dry run)

```
echo "--- on-demand price per hour (Linux, shared tenancy)"
for t in $TYPES; do
  p=$(aws pricing get-products --region us-east-1 --service-code AmazonEC2 --output json --
filters \
      "Type=TERM_MATCH,Field=instanceType,Value=$t"
"Type=TERM_MATCH,Field=location,Value=$LOCATION" \
      "Type=TERM_MATCH,Field=operatingSystem,Value=Linux"
"Type=TERM_MATCH,Field=tenancy,Value=Shared" \
      "Type=TERM_MATCH,Field=preInstalledSw,Value=NA"
"Type=TERM_MATCH,Field=capacitystatus,Value=Used" 2>&1)
  case "$p" in
    *AccessDenied*|*UnauthorizedOperation*|*"not authorized"*) echo "$t: price list not readable
with this IAM user"; continue ;;
  esac
  echo "$p" | python3 -c '
import json, sys
d = json.load(sys.stdin)
for item in d.get("PriceList", []):
    o = json.loads(item)
    for term in o["terms"]["OnDemand"].values():
        for dim in term["priceDimensions"].values():
            print("'"$t"'", dim["pricePerUnit"]["USD"], "USD/h")'
done


echo "--- newest official Debian 13 image"
AMI=$(aws ec2 describe-images --region "$REGION" --owners 136693071363 \
  --filters "Name=name,Values=debian-13-amd64-*" "Name=architecture,Values=x86_64" \
  --query 'sort_by(Images, &CreationDate)[-1].[ImageId, Name, CreationDate]' --output text)
echo "$AMI"


echo "--- dry run of RunInstances (t3.small, nothing is launched)"
aws ec2 run-instances --region "$REGION" --dry-run --image-id "$(echo "$AMI" | cut -f1)" \
  --instance-type t3.small --count 1 2>&1 | tail -1
```

CHAPTER 8: TESTING AND RESULTS

8.1 Testing Strategy

Every claim in this chapter rests on a measurement taken on 31 October 2026. Each benchmark was repeated three times where the environment allowed it; the median is reported and the spread shown. The same Debian 13 user space and OpenSSL 3.5.7 build ran in the container, the KVM guest and the TCG guest, so that only the virtualisation layer differs between them; the host runs Ubuntu's OpenSSL 3.5.5.

8.2 Provisioning a VM

From the `virt-install` call, `cf-app` had an address after **13.9 s**, answered SSH after **15.3 s**, and cloud-init had installed its two packages after **68.2 s** (Figure 8.1). Inside, the guest reports the KVM hypervisor and the host's CPU model; its kernel booted in **3.2 s** and the system was ready 7.3 s later, the remaining time being the package installation.

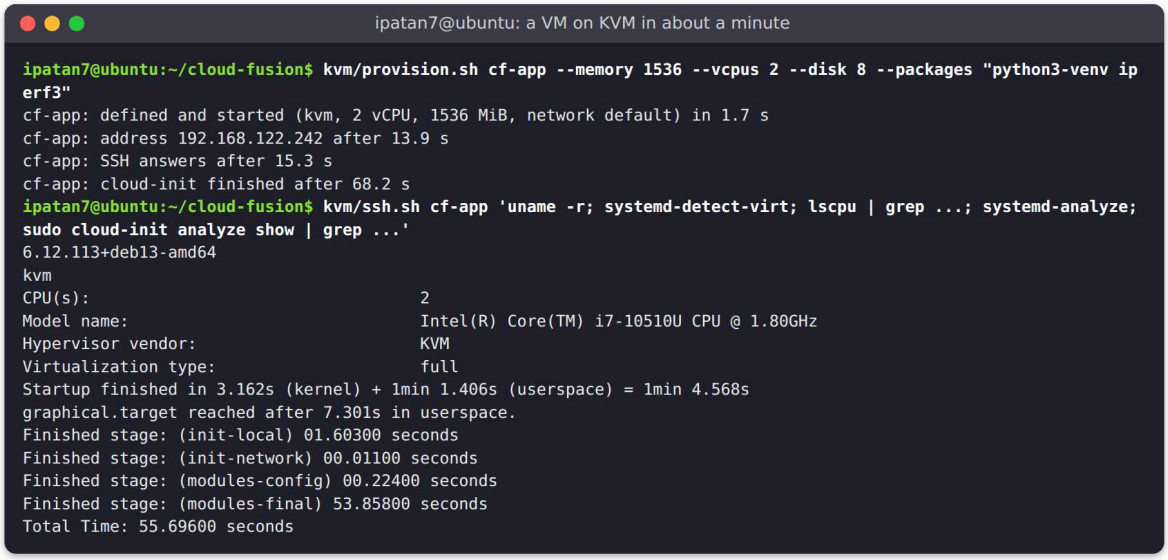


Figure 8.1 – Provisioning cf-app and what the guest sees

Table 8.1 – Provisioning times

VM	Address	SSH	cloud-init done	Packages
cf-app (KVM)	13.9 s	15.3 s	68.2 s	python3-venv, iperf3
cf-peer (KVM)	15.9 s	17.2 s	69.4 s	iperf3
cf-tcg (TCG), run 1	81.0 s	92.4 s	99.0 s	none
cf-tcg (TCG), run 2	73.9 s	83.4 s	88.4 s	none

8.3 Hardware-Assisted and Emulated Virtualisation

The same image without KVM shows what hardware assistance is worth (Figure 8.2). The emulated guest reports `qemu` instead of `kvm` and a generic "QEMU TCG CPU"; its kernel needed **15.9 s** instead of 3.2 s, and SSH answered after 83–92 s instead of 15 s.

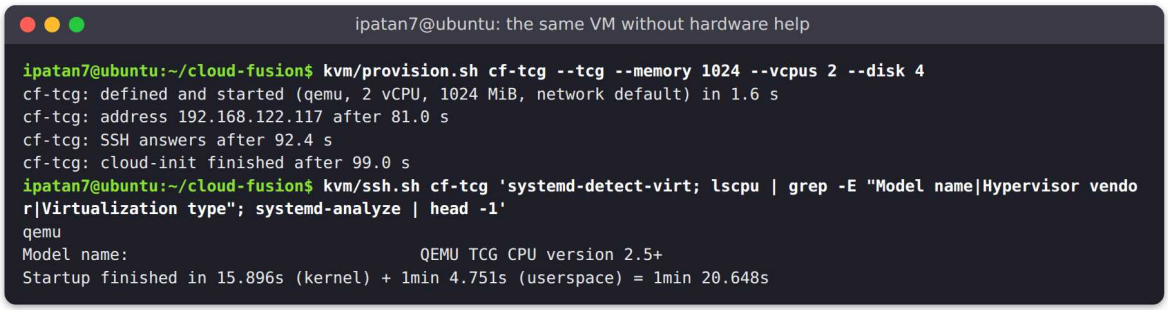


Figure 8.2 – The same VM without hardware assistance

8.4 CPU and Disk Performance

Table 8.2 shows the central result (Figure 8.3). With hardware assistance the guest computes at the speed of the host: the medians of host, container and KVM guest lie within 4 % of each other, less than the spread between runs on one and the same platform (7–13 %; the laptop's turbo frequency varies with temperature). Without hardware assistance the same work runs at **21 %** of the KVM speed, **4.7 times slower**.

Table 8.2 – CPU throughput

Environment	SHA-256, three runs (MB/s)	Median
Host (Ubuntu)	384.6 / 437.1 / 408.3	408.3 MB/s
Container (Debian 13)	429.4 / 419.9 / 386.3	419.9 MB/s
KVM guest (Debian 13)	405.2 / 417.0 / 388.9	405.2 MB/s
TCG guest (Debian 13)	89.7 / 81.3 / 85.4	85.4 MB/s

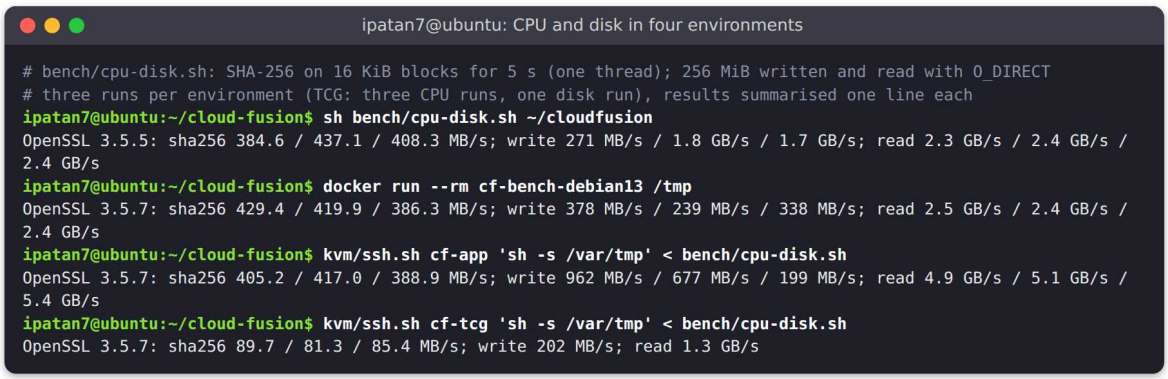


Figure 8.3 – The same CPU and disk measurement in four environments

The disk results need care. The KVM guest *read* at 4.9–5.4 GB/s, more than twice the host's own NVMe (2.3–2.4 GB/s). The guest bypasses its own cache (`O_DIRECT`), but QEMU's default cache mode for the virtual disk is the host's page cache, so the data came from host memory. Writes varied widely on every platform (199 MB/s to 1.8 GB/s), dominated by the SSD's own cache. The honest conclusion is that a virtual disk adds a caching layer that can hide or inflate the physical disk's speed, which matters when sizing storage.

8.5 The Application in a VM and in a Container

CivicSense ran with identical settings – SQLite, two gunicorn workers, 120 demonstration complaints – in the KVM guest and in a Docker container limited to the same two CPUs and 1.5 GB (Figure 8.4). All 4,800 requests succeeded. The VM was slightly faster (78–80 against 72–74 requests per second); the two differed in Python version (3.13.5 in Debian 13, 3.12.16 in the container image) and the container was throttled by its CPU quota, so the result shows that **hardware-assisted virtualisation added no measurable overhead** for this application, not that a VM is faster.

Table 8.3 – CivicSense under load (8 clients, 800 requests per round)

Round	Container	KVM guest	p50 latency (container / VM)
1	74.1 req/s	79.8 req/s	105.7 / 97.3 ms
2	71.8 req/s	78.5 req/s	109.1 / 99.3 ms
3	73.9 req/s	78.4 req/s	106.6 / 99.9 ms

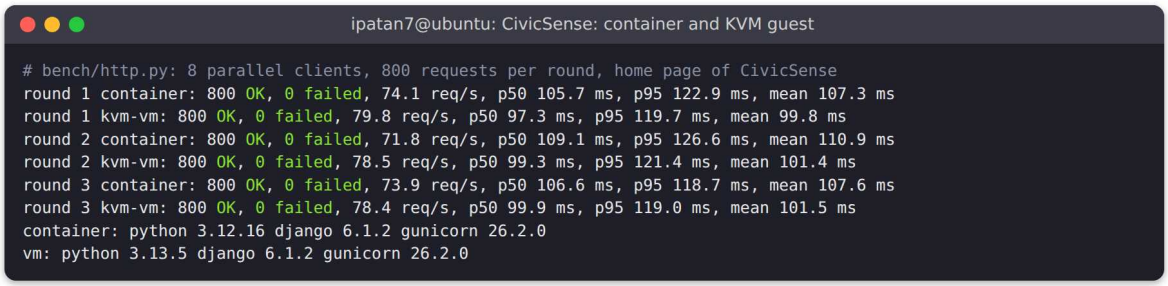


Figure 8.4 – Three rounds against the container and the KVM guest

8.6 Resource Use under Load

During a 30-second load of 2,400 requests (82.5 requests per second, p95 latency 114 ms), `vmstat` inside the guest showed the two vCPUs **94 % busy** (91 % user, 3 % system) while at least 1,159 MiB of memory stayed free; the gunicorn processes used 128 MiB in total, and the whole system 322 MiB when idle. The workload is therefore CPU-bound and light on memory, which decides the instance choice in Section 8.11.

8.7 Storage Virtualisation

Linked clones. After provisioning, the three overlays held 326 MiB (`cf-app`), 362 MiB (`cf-peer`) and 23 MiB (`cf-tcg`, no packages) on top of the shared 419 MiB base: three VMs in about 1.1 GiB instead of three full copies.

Snapshots. A disk-only snapshot of `cf-app` took **0.1 s**: libvirt froze the disk and directed all new writes to a new overlay. A deliberately bad "upgrade" then deleted the application's virtual environment and the site went down (HTTP 000). Reverting to the snapshot and starting the VM brought the application back, answering after **8.9 s** (Figure 8.5). A full snapshot including memory of `cf-peer` was also taken (1.0 s, the image grew from 370 MiB to 1.08 GiB), but its revert failed (Section 8.13).

```
ipatan7@ubuntu: snapshot before a bad upgrade
ipatan7@ubuntu:~/cloud-fusion$ kvm/snapshot-app.sh cf-app
app: HTTP 200
ipatan7@ubuntu:~/cloud-fusion$ virsh snapshot-create-as cf-app before-upgrade --disk-only --atomic
snapshot taken in 0.1 s
 vda   /var/lib/libvirt/images/cf-app.before-upgrade
--- the upgrade goes wrong: the virtual environment is deleted
failed
app: HTTP 000
ipatan7@ubuntu:~/cloud-fusion$ virsh snapshot-revert cf-app before-upgrade && virsh start cf-app
reverted and answering again after 8.9 s
app: HTTP 200
 before-upgrade   2026-10-31 17:11:02 +0530   disk-snapshot
```

Figure 8.5 – A disk-only snapshot before a bad upgrade, and the way back

Thin provisioning and TRIM. The disk benchmark wrote and deleted three 256 MiB files inside `cf-app`, and the image kept the space: it had grown to 1.44 GiB. Deleting a file inside a guest frees nothing on the host unless the guest's TRIM commands reach the image file. With `discard='unmap'` on the virtual disk and `fstrim` in the guest, the image shrank to **772.6 MiB** (Figure 8.6).

```
ipatan7@ubuntu: thin provisioning and TRIM
--- TRIM: deleted files in the guest only free host space if discards reach the image
image allocation before: 1.44 GiB
discard=unmap set on vda
/dev/vda1       7.7G  1.1G  6.3G  15% /
/: 6.6 GiB (7135191040 bytes) trimmed
image allocation after fstrim: 772.60 MiB
cf-app serving: HTTP 200
```

Figure 8.6 – TRIM through the virtual disk returns 700 MiB to the host

8.8 Network Virtualisation

Two VMs on the same virtual bridge exchanged **19.4 Gbit/s** over the NAT network and **17.2 Gbit/s** over the isolated network, with no retransmissions (Figure 8.7) – far beyond the laptop's physical network, because the traffic never leaves host memory. A NIC on the isolated network

was hot-plugged into each running VM and configured in **9.2 s** and **7.8 s**. After the NAT interface of `cf-peer` was unplugged, the VM could no longer resolve or reach any internet address, while it still reached `cf-app` on the isolated network with a round trip of 0.27 ms.

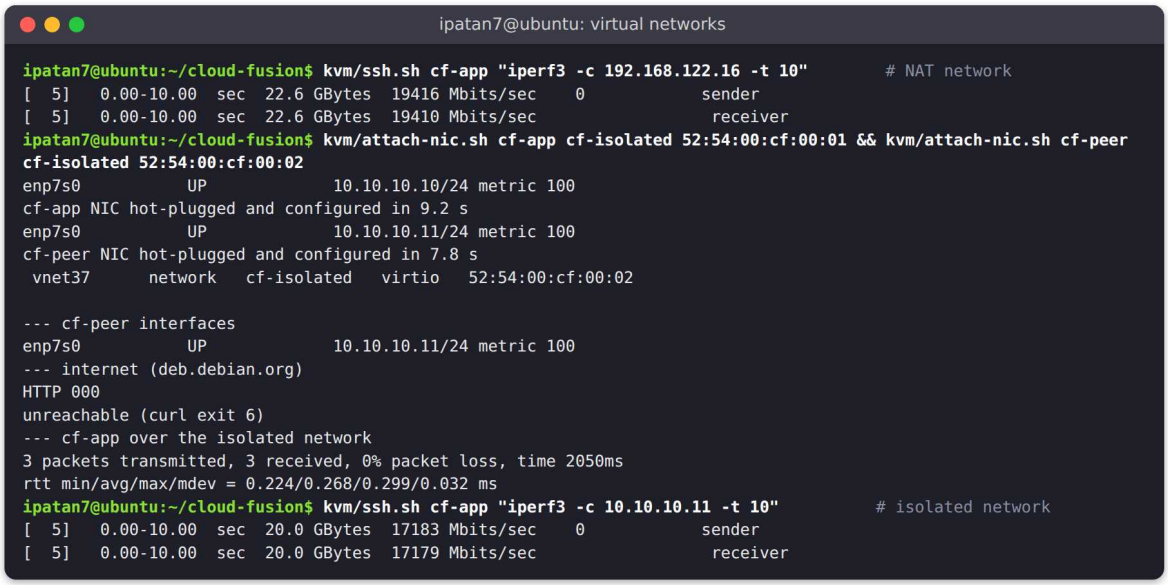


Figure 8.7 – Throughput on both networks, NIC hot-plug and isolation

8.9 Ballooning and Managed Save

The memory balloon shrank the running `cf-app` from 1,469 MiB to 957 MiB as seen by the guest, with the application still answering, and gave the memory back afterwards. `virsh managedsave` wrote the VM's memory and device state to disk and stopped it in **2.8 s**; `virsh start` restored it and the application answered again after **4.6 s**, with the same boot ID – the guest never noticed it had been switched off (Figure 8.8).

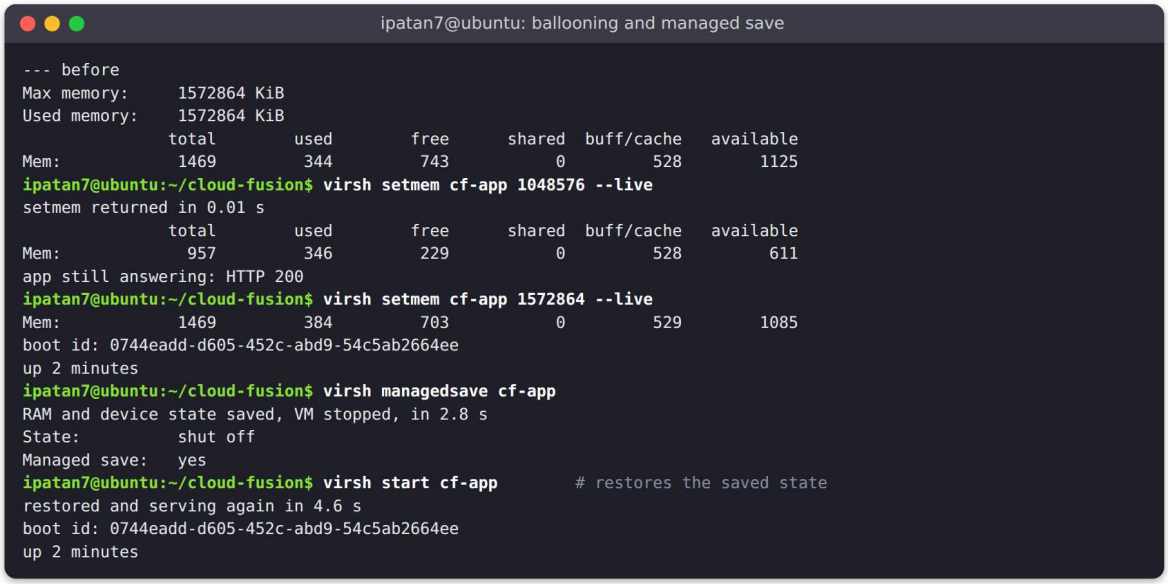


Figure 8.8 – Live memory change and suspend-to-disk

8.10 From KVM to VMware

The disk of `cf-app` (773 MB after TRIM) was copied out of the pool sparsely in 1.1 s and converted with `qemu-img` (Figure 8.9). The **streamOptimized** VMDK, the compressed format of OVA files and AWS VM Import, took 443 MB and 40.6 s; the **monolithicSparse** VMDK, VMware Workstation's usual disk, took 1.2 GB and 2.4 s. VMware's own `vmware-vdiskmanager` reported *Disk chain is consistent* and its check-and-repair finished with status 0. Because VMware Workstation could not start VMs on this kernel, the converted disk was booted with KVM instead: it reached the Debian 13 login prompt, so the converted disk is a working system disk.



Figure 8.9 – Conversion to both VMDK formats, VMware's check and the boot test

Table 8.4 – VMware disk formats of cf-app

Format	Size	Written in	Use
qcow2 (source)	773 MB	–	KVM / QEMU
VMDK streamOptimized	443 MB	40.6 s	OVA export, AWS VM Import
VMDK monolithicSparse	1.2 GB	2.4 s	VMware Workstation (.vmx with pvscsi, vmxnet3)

8.11 Planning the Move to AWS

`aws/plan.sh` read the candidate instance types and their on-demand prices in the Mumbai region from the AWS APIs, found the newest official Debian 13 image – built on 26 October 2026, the same build date as the KVM base image – and dry-ran a launch, which AWS answered with *Request would have succeeded, but DryRun flag is set* (Figure 8.10). Nothing was created and nothing was charged.

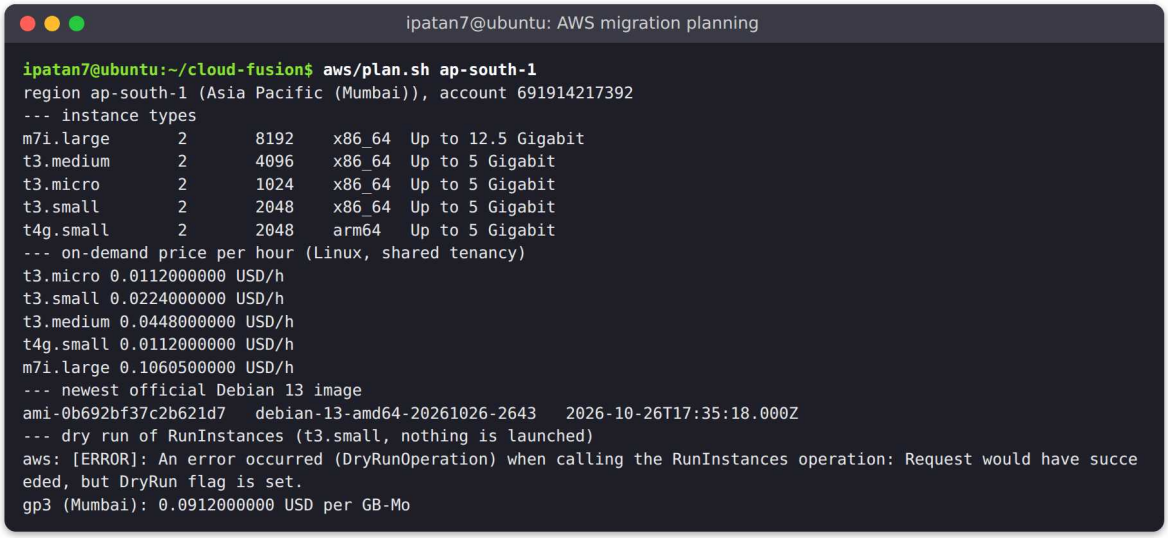


Figure 8.10 – Instance types, Mumbai prices, the Debian AMI and the dry run

Table 8.5 – Candidate EC2 instances (Mumbai, on-demand Linux, read on 31 October 2026)

Type	vCPU	Memory	USD per hour	USD per month (730 h)
t3.micro	2	1 GiB	0.0112	8.18
t3.small	2	2 GiB	0.0224	16.35
t4g.small (arm64)	2	2 GiB	0.0112	8.18
t3.medium	2	4 GiB	0.0448	32.70
m7i.large	2	8 GiB	0.10605	77.42

The measured profile – two busy vCPUs, under 500 MiB of memory – points to a two-vCPU instance with modest memory. **t3.small** (2 vCPU, 2 GiB) is the recommendation: 16.35 USD

a month plus 0.73 USD for an 8 GB gp3 volume (0.0912 USD per GB-month). t3.micro would hold the memory but leaves little room for MySQL in production; t4g.small costs half as much but is an ARM machine, so the x86 disk could not be imported as it is – only the re-platform path would work. t3 instances are burstable, so a load as sustained as the benchmark would eventually be limited by CPU credits; for that, m7i.large is the fixed-performance choice.

Table 8.6 – The seven-step migration model applied to CivicSense

Step	In this project
Assessment	Workload measured: CPU-bound, about 0.3–0.5 GiB memory, 82.5 requests/s on 2 vCPUs
Isolation	Application and its dependencies isolated: one VM, SQLite or MySQL, no host dependencies
Mapping	VM mapped to t3.small; Debian 13 image mapped to the official AMI of the same build
Re-architect	Not needed for a re-host; MySQL (RDS) and S3 for photographs are the next step
Augmentation	cloud-init user-data reused; security group and key pair prepared
Testing	RunInstances dry run passed; the VMDK boot test proved the converted disk
Optimisation	Burstable vs fixed instances weighed against the measured CPU profile

8.12 Summary of Results

Table 8.7 – Summary of measured results

Measurement	Result
New KVM VM to SSH / to configured	15.3 s / 68.2 s
Guest kernel boot, KVM / TCG	3.2 s / 15.9 s
CPU, KVM guest vs host	405 vs 408 MB/s (same within noise)
CPU, TCG guest	85 MB/s: 4.7 times slower than KVM
CivicSense, KVM guest vs container	78–80 vs 72–74 req/s, 0 errors in 4,800 requests
VM-to-VM network (NAT / isolated)	19.4 / 17.2 Gbit/s
Three linked clones	1.1 GiB including the 419 MiB base
Disk-only snapshot / revert to serving	0.1 s / 8.9 s
TRIM through the virtual disk	1.44 GiB → 772.6 MiB
Managed save / restore to serving	2.8 s / 4.6 s, same boot ID
VMDK streamOptimized / monolithicSparse	443 MB / 1.2 GB; consistent; boots
Recommended EC2 instance	t3.small, about 17.08 USD/month with 8 GB gp3

8.13 Defects Found and Fixed

Several problems appeared only when things were actually run. The first cost the most time and is shown in detail (Figure 8.11).

The first VM never booted: GRUB printed *Booting Debian GNU/Linux* and the VM reset, about once a second, without a single kernel message. Changing the CPU model, removing the watchdog, the CD-ROM or the virtio disk changed nothing. Booting the same image with plain QEMU worked every time, so libvirt's exact QEMU command line was replayed outside libvirt with options removed one by one. It looped too, until a VGA card was added. `virt-install --graphics none` creates a VM without any video device, and GRUB in the Debian 13 cloud image resets the machine while it loads the kernel when no video BIOS exists. `provision.sh` now passes `--video vga`.

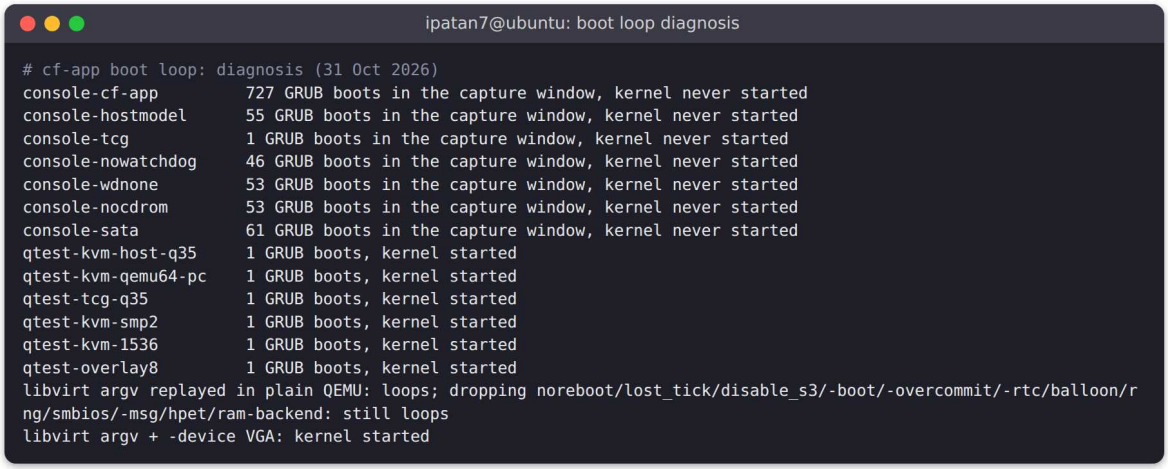


Figure 8.11 – Diagnosing the boot loop: what looped and what booted

Table 8.8 – Defects found and fixed

Defect	Symptom and cause	Fix
VM resets after GRUB	No video device (`--graphics none`); GRUB faults while loading the kernel	`--video vga` in provision.sh
Full snapshot could not be reverted	The snapshot's VM definition named the cloud-init ISO that virt-install had deleted after the first boot	Eject the CD-ROM before snapshots (snapshot-app.sh does it)
Deleted files kept their space	Guest TRIM did not reach the qcow2 file; image grew to 1.44 GiB	`discard='unmap'` + `fstrim`: 772.6 MiB
Overlay merged into the shared base	`blockcommit --active` without `--base` merged into the bottom of the chain; cf-tcg's overlay became invalid	cf-app on its own volume, base re-downloaded; always give `--base`
System disk filled up	`vol-download` wrote the image without holes (1.5 GB)	`--sparse` download; free-space check before each conversion
VMware VMs cannot start	vmmon/vmnet not built for kernel 7.0; building needs root	Disk formats checked with vmware-vdiskmanager and booted on KVM

CHAPTER 9: SCREENSHOTS

The screenshot was taken on 31 October 2026 from the KVM guest `cf-app` at 192.168.122.242:8000; the page is shortened between the list of complaints and the footer. The footer names the release and the slot, which here is the KVM VM.

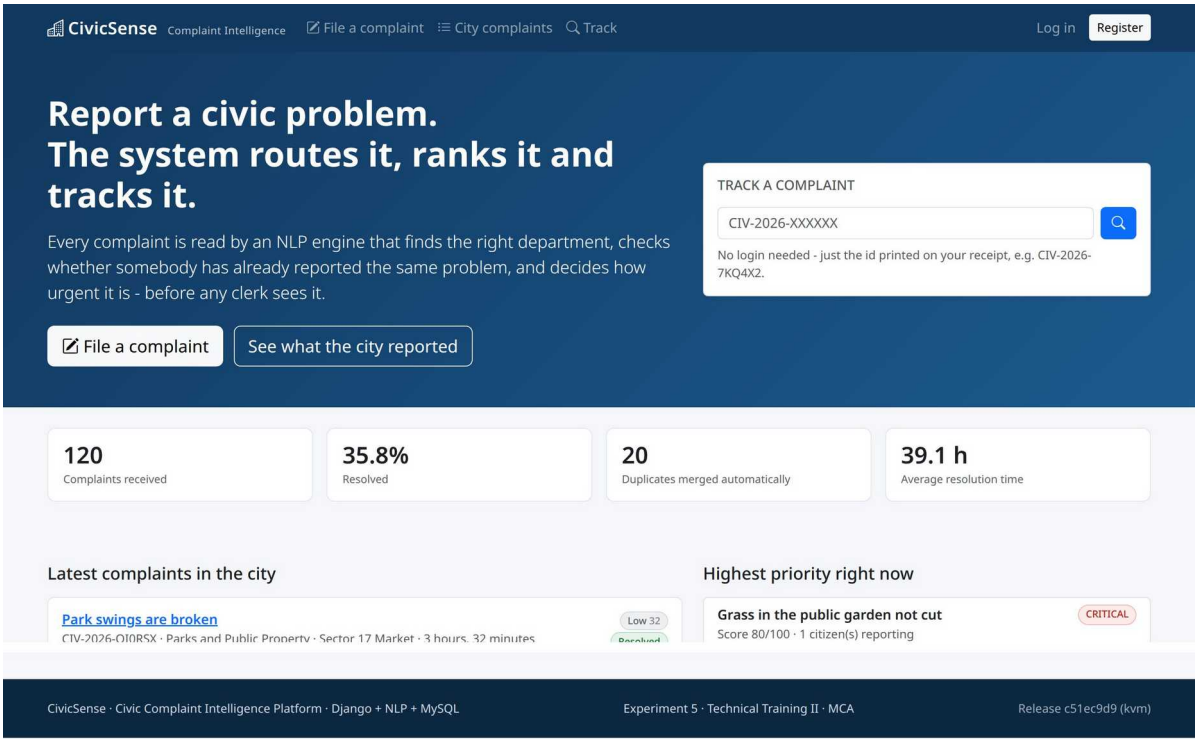


Figure 9.1 – CivicSense served by the KVM virtual machine, with 120 demonstration complaints; the footer reads "Release c51ec9d9 (kvm)"

CHAPTER 10: CONCLUSION

Cloud Fusion moved CivicSense onto virtual machines and connected the on-premises platform with VMware and AWS, with every step scripted and every claim measured. Virtual machines were built from Debian's verified cloud image as linked clones and configured by cloud-init; a new VM answered SSH in about 15 seconds and was fully configured in about a minute. Three VMs fitted in about 1.1 GiB because they share one base image.

The measurements answer the question of what virtualisation costs. With hardware assistance, almost nothing: the KVM guest computed at the host's speed, served CivicSense as fast as a container, and moved 17–19 Gbit/s between VMs. Without hardware assistance, a lot: the emulated guest ran 4.7 times slower and booted five times slower – the reason VT-x made virtualisation practical. The virtual disk turned out to be the subtle part: its cache can make reads look faster than the physical disk, and thin images only give space back when TRIM is passed through.

The work also showed the operations that make virtual machines valuable: a disk-only snapshot taken in 0.1 s turned a broken upgrade into a nine-second rollback, a running VM gave back half a gigabyte of memory, joined an isolated network and was suspended and resumed without noticing. Its disk was converted into VMware's formats, accepted by VMware's tool and booted again; and the move to AWS was planned with real prices and a successful dry run, leading to a t3.small at about 17 USD a month.

The defects – a boot loop caused by a missing video device, a snapshot that referred to a deleted file, a merge into the wrong image, a disk filled by a non-sparse copy – were each diagnosed from evidence and fixed in the scripts. They are the practical lessons of the project: virtual infrastructure behaves like real infrastructure, and has to be tested like it.

CHAPTER 11: FUTURE SCOPE

- **Run the VMware side** – build the VMware kernel modules (or use an ESXi host) and start the converted VM in VMware itself.
- **Execute the AWS migration** – import the streamOptimized VMDK with VM Import, or launch the Debian AMI with the same cloud-init file, and repeat the benchmarks on EC2.
- **Live migration** – a second KVM host with shared storage, to move a running VM with `virsh migrate --live` and measure the pause.
- **Storage tuning** – compare cache modes (none, writeback) and raw versus qcow2 disks with a proper I/O tool such as fio.
- **Infrastructure as code** – describe the VMs, networks and the EC2 instance with Terraform so that both worlds come from one definition.
- **Hybrid networking** – a VPN between the isolated libvirt network and an AWS VPC, the classic hybrid-cloud setup.

REFERENCES

[1] A. Kivity, Y. Kamay, D. Laor, U. Lublin and A. Liguori, "kvm: the Linux Virtual Machine Monitor", Proceedings of the Linux Symposium, Ottawa, 2007.

[2] F. Bellard, "QEMU, a Fast and Portable Dynamic Translator", USENIX Annual Technical Conference, 2005.

[3] G. J. Popek and R. P. Goldberg, "Formal Requirements for Virtualizable Third Generation Architectures", Communications of the ACM, vol. 17, no. 7, 1974.

[4] R. Uhlig et al., "Intel Virtualization Technology", IEEE Computer, vol. 38, no. 5, 2005.

[5] R. Buyya, J. Broberg and A. Goscinski (eds.), Cloud Computing: Principles and Paradigms, Wiley, 2011 (chapter on migrating into a cloud: the seven-step model).

[6] libvirt project, "Domain XML format", "Storage management", "Network XML format" and "Snapshot XML format", https://libvirt.org/docs.html (accessed October 2026).

[7] QEMU project, "Disk Images" and "qemu-img", https://www.qemu.org/docs/master/.

[8] Canonical, "cloud-init documentation: NoCloud datasource", https://cloudinit.readthedocs.io/.

[9] Debian Cloud Team, "Debian Official Cloud Images", https://cloud.debian.org/images/cloud/.

[10] VMware, "Virtual Disk Format 1.1" (VMDK specification) and "Using the Virtual Disk Manager Utility", VMware documentation.

[11] Amazon Web Services, "Amazon EC2 instance types", "VM Import/Export User Guide" and "AWS Price List Query API", https://docs.aws.amazon.com/.

[12] M. Portnoy, Virtualization Essentials, 2nd edition, Sybex, 2016.

[13] M. I. Patan, "civicsense-devops: CivicSense delivered automatically", GitHub, 2026, https://github.com/iPatan7/civicsense-devops.

[14] M. I. Patan, "cloud-fusion: one application across KVM, VMware and AWS", GitHub, 2026, https://github.com/iPatan7/cloud-fusion.

APPENDIX A: RUNNING AND DEMONSTRATING THE PROJECT

The steps below rebuild the setup on a Linux machine with KVM, libvirt, virt-install and qemu-img; the user must be in the `libvirt` and `kvm` groups.

A.1 Base image and the first VM

```
git clone https://github.com/iPatan7/cloud-fusion.git && cd cloud-fusion
mkdir -p ~/cloudfusion/keys && ssh-keygen -t ed25519 -N '' -f ~/cloudfusion/keys/id_ed25519
kvm/fetch-base.sh
kvm/provision.sh cf-app --memory 1536 --vcpus 2 --packages "python3-venv iperf3"
app/deploy-civicsense.sh cf-app           # then open http://<address>:8000/
```

A.2 Compare with emulation and measure

```
kvm/provision.sh cf-tcg --tcg
kvm/ssh.sh cf-app 'sh -s /var/tmp' < bench/cpu-disk.sh
kvm/ssh.sh cf-tcg 'sh -s /var/tmp' < bench/cpu-disk.sh
python3 bench/http.py http://<cf-app address>:8000/ --clients 8 --requests 800
```

A.3 Networks, snapshots and live operations

```
virsh net-define kvm/cf-isolated.xml && virsh net-start cf-isolated
kvm/attach-nic.sh cf-app cf-isolated 52:54:00:cf:00:01
kvm/snapshot-app.sh cf-app                # snapshot, bad upgrade, revert
virsh setmem cf-app 1048576 --live         # balloon to 1 GiB
virsh managedsave cf-app && virsh start cf-app
```

A.4 VMware formats and the AWS plan

```
convert/to-vmware.sh cf-app               # VMDKs checked and deleted again; KEEP=1 keeps
them
aws/plan.sh ap-south-1                    # read-only: types, prices, AMI, dry run
```